InterviewIQ

AI Interview Simulator

Product Requirements Document · v2.0

Author	Kiran Sindam
AI Mentor	Claude (Anthropic)
Version	2.0
Date	8 March 2026
Category	Career Tech / Developer Tools
Status	Active Development

Built with by Kiran Sindam · Mentored by Claude (Anthropic)

1. Executive Summary

InterviewIQ is a Streamlit-based AI interview simulator that transforms a candidate's resume into a fully personalised mock interview session. It solves a high-impact, underserved problem: the gap between technical ability and interview communication skills costs developers job offers daily.

The tool parses PDF/TXT resumes to extract skills, projects, and experience, then generates three categories of questions — Technical, Project-based, and Behavioural — tailored to the specific candidate. Every answer is scored 1-5 with a targeted improvement tip.

2. Problem Statement

The Gap Between Skill and Communication

Technical recruiters spend fewer than 90 seconds on initial screening. In that window, candidates who can't articulate their experience clearly are eliminated — regardless of their actual ability. Studies across FAANG and mid-size tech companies consistently show that failed interviews stem from poor communication as often as insufficient technical knowledge.

Four Specific Pain Points

- Generic prep tools give every user the same questions, ignoring their actual resume and background.
- Practicing alone — in a mirror or with friends — provides no objective, scored feedback.
- Developers can build impressive projects but struggle to articulate architecture decisions, trade-offs, and their personal impact during interviews.
- The STAR method (Situation, Task, Action, Result) is widely taught but rarely practised with real feedback loops.

Who Is Affected Most

Segment	Pain Point	InterviewIQ Solution
CS Students	No structured feedback on personal projects	Instant personalised interview audit
Bootcamp Graduates	Projects seem unimpressive to reviewers	Project-specific question generation
Junior Engineers	Cannot gauge what good answers look like	1-5 score + targeted improvement tip
Tech Mentors	Manual mock interviews are time-consuming	Structured AI session in under 2 minutes

3. Solution Architecture

Module 1: resume_parser.py

The resume intelligence layer. Accepts PDF (via PyPDF2 with pdfminer fallback) and TXT files. Splits the resume into named sections (Skills, Experience, Projects, Education) using regex section detection. Skill extraction uses a 50+ entry curated database across 8 technology categories with whole-word lookahead/lookbehind matching. Project extraction uses four complementary patterns to handle bullet lists, explicit project headers, pipe-delimited entries, and numbered lists.

Module 2: interview_engine.py

The core interview logic. Generates a personalised question set by sampling Technical templates (filled with detected skills), Project templates (filled with detected project names), and a random selection of HR/Behavioural questions. Questions are intelligently interleaved — Technical, HR, Project cycling — to match the rhythm of a real interview. The answer evaluator scores on a 1-5 rubric using word count thresholds, a 25-keyword impact word list, STAR method signal detection, and quantification detection (numbers with units).

Module 3: app.py (Streamlit UI)

The presentation layer. Fully decoupled from business logic. Renders a premium dark-mode interface using custom injected CSS with Sora and JetBrains Mono typography, gradient animations, progress bars, category-tagged question cards, colour-coded feedback blocks, and a session completion dashboard with score breakdown grid.

Data Flow

```
Resume file upload
    └─> resume_parser.py   (extract: skills, projects, experience)
    └─> interview_engine.py (generate: personalised question set)
    └─> app.py             (render: interactive interview UI)
    └─> interview_engine.py (evaluate: score + tip per answer)
    └─> app.py             (display: session score dashboard)
```

4. Feature Specification

Feature	Description	Priority	Status
Resume Parsing	50+ skills, 4 project patterns, section splitting	P0	Done
Personalised Questions	Technical, Project, HR from actual resume data	P0	Done

Feature	Description	Priority	Status
1-5 Answer Scoring	Rubric: word count + keywords + STAR + numbers	P0	Done
Targeted Feedback Tips	Gap-specific improvement per answer	P0	Done
Session Dashboard	Avg score, breakdown grid, question review	P1	Done
Interleaved Question Flow	Tech/HR/Project cycling for realism	P1	Done
Progress Bar	Live X/N question progress indicator	P1	Done
Word Count in Feedback	Shows word count for self-awareness	P2	Done
Claude API Scoring	LLM-powered semantic answer evaluation	P1	Planned
Voice Input	Speak answers via browser microphone	P2	Planned
PDF Report Export	Downloadable session analysis report	P2	Planned

5. Code Improvements: v1 vs v2

Area	v1 Original	v2 Improved
Critical Bug	Regex \b inside re.escape() — zero skills detected	Lookahead/lookbehind pattern — correct detection
Skills DB	11 hardcoded skills	50+ across 8 tech categories
Project Extraction	1 pattern: 'Project:' prefix only	4 patterns: bullets, headers, pipes, numbered
Answer Scoring	Word count only	Word count + 25 keywords + STAR + numbers
Question Order	Random shuffle of all questions	Interleaved by category for realism
Feedback Tips	Generic by score only	Gap-targeted per detected keyword absence
Score Breakdown	Not present	4-bucket grid: 5/4/3/1-2 star counts
Error Handling	Single PDF library, no fallback	PyPDF2 with pdfminer.six fallback, multi-encoding
Separation of Concerns	Mixed logic in some functions	Fully decoupled: UI / engine / parser

6. UI / UX Design System

Design Philosophy

The interface was designed to feel like a professional developer tool — not a student project. The aesthetic direction is 'Dark Tech with Editorial Precision': maximum contrast, terminal-inspired typography, smooth micro-animations, and colour as information (not decoration).

Typography

- Display / UI: Sora (800 weight) — modern, technical, distinctive
- Code / labels / metadata: JetBrains Mono (400/600) — programmer-native readability

Colour System

Token	Value	Usage
--bg	#070b12	Page background
--cyan	#00d4ff	Primary accent, technical category
--indigo	#6366f1	Buttons, active states, project category
--green	#10b981	Success, 5-star feedback
--amber	#f59e0b	Warning, 3-star feedback
--red	#ef4444	Error, 1-2 star feedback

Animation Principles

- fadeUp keyframe: every new content block slides up 18px over 0.55s — creates perceived page freshness
- pop keyframe: hero icon bounces in with scale spring — creates a memorable first impression
- Progress bar uses CSS width transition for smooth fill updates
- Feedback blocks use score-specific background classes — colour communicates quality without reading

7. Product Roadmap

Phase 2 — AI Scoring (Next Quarter)

- Integrate Claude API to replace heuristic scoring with semantic LLM evaluation
- Claude reads the full question and answer together for context-aware scoring
- Generate personalised interview talking points for the candidate's strongest projects

Phase 3 — Platform Features

- Voice input via browser Web Speech API — speak your answers naturally
- Company-specific interview modes: Google (system design focus), Amazon (leadership principles), startups
- Commit history import: analyse GitHub activity alongside resume skills

Phase 4 — Growth

- User accounts with session history and progress tracking over time
- PDF export of full session analysis report
- Shareable public analysis links for mentor review

8. Installation Guide

Prerequisites

- Python 3.9 or higher
- pip package manager
- A public GitHub account (for version control and deployment)

Step-by-Step

Step 1 — Clone the repository:

```
git clone https://github.com/kiransindam/InterviewIQ-AI-Interview-Simulator
```

Step 2 — Install dependencies:

```
pip install streamlit PyPDF2 pdfminer.six
```

Step 3 — Launch the application:

```
streamlit run app.py
```

Step 4 — Open your browser to http://localhost:8501 and upload your resume to begin.

9. Author & Credits

Developer

Name: Kiran Sindam

Role: Creator, Full-Stack Developer, and Data Scientist , Data Analytics

GitHub: github.com/kiransindam

LinkedIn: linkedin.com/in/kiransindam

AI Project Mentor

Name: Claude, by Anthropic

Contributions in v2.0: Architecture review and separation of concerns refactor; identification of the critical regex escape bug in resume_parser.py; expansion of the skills database from 11 to 50+ entries; design of the interleaved question flow; UI design system specification; and authorship of this full PRD document.

License

MIT License. Free to use, fork, and build upon with attribution.